

Дълбоко обучение с подкрепление за настолна игра "Заселниците на Катан"

Александър Македонски
Факултет по математика и информатика
Софийски университет „Св. Климент Охридски“
alemak2002@gmail.com

24.06.2026

Абстракт

Представям система за дълбоко обучение с подкрепление (DRL), предназначена да се научи да играе Заселниците на Катан — сложна многоигрочова настолна игра, която комбинира стохастично генериране на ресурси, преговори и дългосрочно стратегическо планиране. Имплементирам и сравнявам три алгоритъма за DRL — Deep Q-Network (DQN), Proximal Policy Optimisation (PPO) с Generalised Advantage Estimation (GAE) и базова линия REINFORCE — използвайки средата *catanatron gym*. Ключов принос е плътна, оформена функция за награда, която декомпозира оскъдния краен сигнал на победа в стъпкови междинни награди за построяване на пътища, селища и градове, както и термин за поставяне на разбойника, който награждава враждебността към опоненти. Моят най-добър агент — Dueling Double DQN (D3QN), трениран чрез самостоятелна игра — постига 90% на победи срещу базовата линия на (претеглен) случаен играч. Стандартен DQN с оформена награда достига 85%, а PPO с бинарна награда победа/загуба — 76%. Анализираме влиянието им върху тренировката. Допълнително: Уеб-базиран UI позволява на хора да се изправят срещу тренираните модели. Отворен код.

1 Въведение

Настолните игри отдавна служат за ориентир в изследванията в областта на изкуствения интелект — от дама и шах [Samuel, 1959, Silver et al., 2017] до Го [Silver et al., 2016]. Заселниците на Катан представлява особено интересно предизвикателство, тъй като съчетава стохастични елементи (хвърляне на зарове на всеки ход, случайна начална наредба на средата), голямо дискретно пространство от действия (над 300 допустими действия на стъпка), дълги епизоди (стотици ходове на игра) и многоигрочова конкурентна динамика. Тези свойства правят стандартни DRL подходи нетривиални.

Съществуващите изследвания за ИИ в Катан са разглеждали Monte Carlo Tree Search и евристики, базирани на правила [Szita et al., 2010], но подходите с дълбоко RL остават слабо проучени.

В настоящата работа изследваме дали съвременните DRL алгоритми могат да научат конкурентна игра на Катан от нулата. Нашите основни приноси са:

1. Плътна функция за оформяне на наградата, която декомпозира сигнала за победа в интерпретируеми компоненти за отделни събития, включително нов термин за поставяне на разбойника, който различава нараняването на опоненти от наказването на самия агент.
2. Емпирично сравнение на DQN (стандартен и Dueling), PPO с GAE и REINFORCE в двуигрочова постановка на Катан.

3. Анализ на това как тренировката чрез самостоятелна игра (сам срещу себе си) подобрява представянето на DQN.
4. Допълнение - Уеб-базиран UI за игра човек срещу агент, изграден върху визуализатора на catanatron.

2 Свързани изследвания

ИИ за Катан. [Szita et al. \[2010\]](#) прилагат Monte Carlo Tree Search за Катан, постигайки приемлива игра, но разчитайки на ръчно изградена политика за разгръщане. [Pfeiffer \[2004\]](#) изследват агенти, базирани на правила. По-наскоро рамката catanatron [\[Collazo, 2021\]](#) предоставя среда, съвместима с gym, и базова линия с претеглен случаен играч. По наши данни никое предходно изследване не е публикувало резултати от DRL тренировка от край до край в многоигровата gym постановка.

Deep Q-Networks. Алгоритъмът DQN [\[Mnih et al., 2015\]](#) - повторение на опита, целева мрежа за стабилизиране. Dueling DQN [\[Wang et al., 2016\]](#) — стойност на състоянието $V(s)$ и предимство с нулево средно $A(s, a)$, което е особено полезно когато много действия имат подобна очаквана възвращаемост. Double DQN [\[Hasselt et al., 2016\]](#) - адресира надценяването на Q-стойностите.

Методи на градиента на политиката. PPO [\[Schulman et al., 2017\]](#) — предлага и по-стабилна тренировка от обикновения градиент на политиката. GAE [\[Schulman et al., 2015\]](#) интерполира между Монте Карло оценки и едностъпкови TD оценки, подходящ за дълги игри.

Самостоятелна игра. Ние приемаме опростена версия, при която замразено копие на текущия най-добър модел служи като опонент, позволявайки на агента да се подобрява срещу нестационарен, но все по-силен противник.

3 Метод

3.1 Среда

Използваме gym средата catanatron (catanatron/Catanatron-v0). Наблюдението е плосък вектор с размер $194N + 226$, където N е броят играчи. За двуйгровата игра това дава 614-мерно наблюдение. Пространството от действия е дискретно множество от $|\mathcal{A}| = 328$ цели числа. На всяка стъпка средата връща и множество от допустими действия; недопустимите действия се маскират с малка стойност, за да не може агентът да ги избере.

3.2 Оформена функция за награда

Наградата по подразбиране на средата е бинарна (+1 за победа, -1 за загуба). Дефинираме междинна оформена награда r_t , която допълва този сигнал:

$$r_t = r_t^{\text{точки}} + r_t^{\text{път}} + r_t^{\text{град}} + r_t^{\text{селище}} + r_t^{\text{победа}} \quad (1)$$

където всеки термин е дефиниран по следния начин. (Δ означава разликата между сегашното наблюдение и предходното.) $r_t^{\text{точки}} = \max(0, \Delta_{\text{точки}_t}) \cdot c_{\text{свр}}$, където $c_{\text{точки}} = 3$ Награждават се само положителни печалби на ТП — временни загуби (îäð. загуба на Най-дълъг път) не се наказват, за да не се обезкуражи строенето на пътища; $r_t^{\text{път}} = \max(0, \Delta_{\text{пътища}_t}) \cdot c_{\text{път}}$, където $c_{\text{път}} = 0, 2$; $r_t^{\text{град}} = \max(0, \Delta_{\text{градове}_t}) \cdot c_{\text{град}}$, където $c_{\text{град}} = 1, 5$; $r_t^{\text{селище}} = \max(0, \Delta_{\text{селища}_t} - 2) \cdot c_{\text{селище}}$, където $c_{\text{селище}} = 1$; Награждават се само селища, построени след задължителните две начални; и

$$r_t^{\text{победа}} = \begin{cases} +20 & \text{ако агентът печели на стъпка } t \\ -20 & \text{ако опонентът печели на стъпка } t \\ 0 & \text{иначе} \end{cases}$$

По-късно се представя и друга награда, която за съжаление не влиза в тестовите — $r_t^{\text{разбойник}}$:

Поставяне на разбойника. Когато агентът поставя разбойника, оценяваме качеството на избраната плочка:

$$r_t^{\text{разбойник}} = c_p \sum_{v \in \mathcal{N}} \begin{cases} c_{\text{опонент}} \cdot f_v & \text{ако сградата при } v \text{ е на опонент} \\ c_{\text{собствено}} \cdot f_v & \text{ако сградата при } v \text{ е на агента} \end{cases} \quad (2)$$

където $\mathcal{N}$ е множеството от шестте върха около плочката с разбойника, $f_v = c_{\text{град_фактор}}$ ако има град и 1 в противен случай, $c_p = 0,4$, $c_{\text{опонент}} = 1,2$, $c_{\text{собствено}} = -1$ и $c_{\text{град_фактор}} = 2$. Асиметрията $c_{\text{опонент}} > c_{\text{собствено}}$ предава идеята, че нараняването на опонент е по-ценно от защитата на себе си.

3.3 Архитектура на мрежите

Всички мрежи използват една и съща начална архитектура:

$$\mathbf{h} = \text{ReLU}(W_3 \text{ReLU}(W_2 \text{ReLU}(W_1 \mathbf{s})))$$

Някои модели имплементират допълнителни слоеве над нея или я разширяват с по-голям брой неврони (в повечето случаи двоен).

3.4 Архитектура на Dueling DQN

Вариантът Dueling DQN разделя крайното представяне на два клона:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \quad (3)$$

Това помага, когато много действия имат подобно предимство, което е честа ситуация в Катан (напр. 50% от състоянията са „край на хода които дават 0 точки).

За тренировка се използва загубата на Хубер между предсказаните и целевите Q-стойности:

$$\mathcal{L} = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\delta \left(Q(s, a) - \left(r + \gamma \max_{a'} \hat{Q}(s', a') \right) \right) \right] \quad (4)$$

където $\hat{Q}$ е целевата мрежа и $\delta(\cdot)$ е загубата на Хубер. Недопустимите действия се маскират до $-\infty$ преди операцията $\max$ за избиране на действие в средата.

3.5 PPO с GAE

Актьорът и критикът на PPO са отделни MLP-та, споделящи същата скрита архитектура. Отрязаната сурогатна цел е:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right] \quad (5)$$

Предимствата $\hat{A}_t$ се оценяват чрез GAE с $\lambda \in \{0,90, 0,95\}$ (претърсвано по време на тренировка).

3.6 Самостоятелна игра

Имплементираме самостоятелна игра за DQN, където агентът е изправен срещу свое копие, което се обновява периодично. Така агентът се подобрява заедно с опонента, предотвратявайки прекалено нагаждане към фиксираната базова линия.

4 Експериментална постановка

4.1 Среда и базови линии

Всички експерименти използват catanatron/Catanatron-v0 gym в двуигрочова конфигурация. Основната базова линия е WeightedRandomPlayer от catanatron, който избира действия с ръчно настроени евристични тегла. Равномерен RandomPlayer се използва като по-слаба референция.

4.2 Данни и събиране на епизоди

Няма офлайн набор от данни; всички тренировъчни данни се генерират онлайн чрез самостоятелна игра или игра срещу базови линии. Всеки епизод съответства на една пълна игра на Катан (обикновено 100–400 хода). Тренировъчните цикли варират от 5 000 до 20 000 епизода.

4.3 Хиперпараметри

Таблица 1: Ключови хиперпараметри за всеки алгоритъм.

Параметър	DQN / D3QN	PPO
Оптимизатор	Adam	Adam
Скорост на учене	1×10^{-4}	3×10^{-4}
Отстъпка γ	0,99	0,99
Размер на пакет	128	пълнен rollout
Памет за повторение	100 000	—
ϵ начало/край	1,0 / 0,05	—
Затихване на ϵ	1×10^{-6} /стъпка	—
Синхр. на целева мрежа	на всеки 5 епизода	—
GAE λ	—	0,95
PPO clip ϵ	—	0,2
Коеф. на ентропия	—	0,01

4.4 Метрика за оценка

Основната метрика е процентът на победи за 500 оценъчни епизода срещу WeightedRandomPlayer, изиграни след тренировката. Вторичните метрики са средната дължина на играта (брой ходове), общата награда за епизод и средната загуба за епизод.

Създадени са и допълнителни оценки за игри изиграни срещу фиксирани играчи от catanatron (Alpha Beta Pruning или MCTS алгоритми.)

4.5 Хардуер и софтуер

Експериментите са проведени на единична потребителска графична карта. Всички модели са имплементирани в PyTorch. Средата е catanatron v $\geq$ 1.0 [Collazo, 2021], а взаимодействията с gym използват API-я на gymnasium. Тренировъчните статистики се записват като JSON и се визуализират чрез персонализирано Streamlit табло.

5 Резултати

5.1 Сравнение на процента на победи

Таблица 2: Процент на победи срещу WeightedRandomPlayer (двуигрочова игра). По-висок е по-добър; 50% съответства на игра на ниво случаен избор.

Алгоритъм	Награда	Епизоди	Победи (%)
Случайна базова линия	—	—	~50
DQN (стандартен)	Оформена	10 000	85
D3QN + самост. игра	Оформена	15 000	90
PPO + GAE	Победа/загуба (± 1)	20 000	76
REINFORCE	Оформена	10 000	25

5.2 Ефект на самостоятелната игра

Замяната на фиксирания опонент WeightedRandomPlayer с замразено копие на текущия DQN модел увеличава крайния процент на победи от 85% до 90% при 15 000 епизода. Това предполага, че адаптивната програма, генерирана от самостоятелната игра, осигурява по-трудни и разнообразни опоненти, подобрявайки обобщаването.

6 Аблационни изследвания

Таблица 3: Аблационно изследване на компонентите на наградата за DQN (10 000 епизода).

Конфигурация	Победи (%)
Пълна оформена награда	85
— награда за пътища	+2 пп ()
— термини за градове/селища	+10 пп ()
— термин за разбойника	все още не е тествано
Само бинарна победа/загуба	≈ 50 (без подобрение)
Стандартен DQN $\rightarrow$ Dueling DQN	+5 (от 85 до 90 при самост. игра)
Фиксиран опонент $\rightarrow$ самостоятелна игра	+5 (от 85 до 90)

Преминаването от стандартен DQN към Dueling архитектура подобри стабилността на тренировката, в съответствие с хипотезата, че много действия в Катан имат подобни стойности на награда и оценката на състоянието трябва да бъде отделена. Бинарната награда победа/загуба самостоятелно беше недостатъчна: тя не доведе до подобрение, потвърждавайки необходимостта от плътно оформяне при дълги игри.

7 Дискусия и ограничения

7.1 Критични бъгове

Поради непредвидени бъгове в ръчно написан код, се отдели много време в трениране на безполезни агенти. Дори при писане на тестове на кода, бъгът остана скрит, тъй като беше недетерминистичен. За повече информация, може да се види дневника на проекта, където са описани всички стъпки на развитието на проекта.

7.2 Ограничения

- Всички агенти се оценяват в двуигрочова постановка. Четириигрочовата постановка беше изследвана накратко.
- Векторът на наблюдение е плосък масив от характеристики, който не кодира изрично

пространствената структура на дъската. Граф невронна мрежа или CNN върху тензора на дъската биха уловили локалността по-добре, но този метод не бе изследван.

- REINFORCE се представи изключително слабо; не изследвахме напълно дали това се дължи на бърз в тренировката или на алгоритмично ограничение.
- Търсенето на хиперпараметри беше ръчно и ограничено. Систематично претърсване (напр. Байесова оптимизация) би могло да даде допълнителни подобрения.
- Скоростта на тренировка беше: приблизително 2000 епизода за 30 минути на единична GPU.

8 Бъдеща работа.

Обещаващи посоки включват (1) прилагане на граф невронни мрежи върху графовата структура на дъската, (2) пълна четириигрочова самостоятелна игра с population-based тренировка, (3) интегриране на вероятността за производство на ресурси в наблюдението за подобряване на решенията за поставяне на селища, и (4) система за търговия между играчите, изпълняваща се от отделна невронна мрежа.

Литература

- Collazo, B. (2021). Catanatron: A Catan engine and OpenAI gym environment. <https://github.com/bcollazo/catanatron>.
- van Hasselt, H., Guez, A., and Silver, D. (2016). Deep reinforcement learning with double Q-learning. In AAI, pages 2094–2100.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Pfeiffer, M. (2004). Reinforcement learning of strategies for Settlers of Catan. In *Proceedings of the International Conference on Computer Games*.
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Silver, D., Huang, A., Maddison, C. J., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489.
- Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*.
- Szita, I., Chaslot, G., and Spronck, P. (2010). Monte-Carlo tree search in Settlers of Catan. In *Advances in Computer Games*, pages 21–32.
- Wang, Z., Schaul, T., Hessel, M., Lanctot, M., Freitas, N., and Silver, D. (2016). Dueling network architectures for deep reinforcement learning. In *ICML*, pages 1995–2003.